PAYMENT MANAGEMENT SYSTEM

DBMS Project Documentation • Based on the supplied MySQL script

Contents

1. Payment Table Design
2. Order–Payment ER Diagram
3. Sample Customer, Order and Payment Data
4. Payment Transaction & Revenue Analysis
5. Query Output Screenshots (Workbench-style sample results)

Note: The supplied SQL script creates and inserts records only into the **Payment** table. It references existing **Orders** and **Customer** tables but does not include their CREATE TABLE statements or insert statements. Therefore, Customer/Order join values below are illustrative; Payment records and query logic are taken from the supplied script.

1. Payment Table Design

The Payment table stores the payment information for each order. Each order can have at most one payment record because Order_ID is UNIQUE.

Column	Data type	Constraint / purpose
Payment_ID	INT	PRIMARY KEY, AUTO_INCREMENT
Order_ID	INT	NOT NULL, UNIQUE, FOREIGN KEY references Orders(Order_ID)
Payment_Date	DATE	NOT NULL
Payment_Mode	VARCHAR(30)	NOT NULL; UPI, Credit Card, Debit Card, Net Banking, Cash on Delivery
Payment_Status	VARCHAR(20)	NOT NULL, DEFAULT 'Pending'; Pending, Success, Failed
Transaction_Amount	DECIMAL(10,2)	NOT NULL; CHECK amount >= 0

SQL table creation

```
CREATE TABLE Payment (
  Payment_ID INT PRIMARY KEY AUTO_INCREMENT,
  Order_ID INT NOT NULL UNIQUE,
  Payment_Date DATE NOT NULL,
  Payment_Mode VARCHAR(30) NOT NULL,
  Payment_Status VARCHAR(20) NOT NULL DEFAULT 'Pending',
  Transaction_Amount DECIMAL(10,2) NOT NULL,
  FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID),
  CHECK (Payment_Mode IN
    ('UPI','Credit Card','Debit Card','Net Banking','Cash on Delivery')),
  CHECK (Payment_Status IN ('Pending','Success','Failed')),
  CHECK (Transaction_Amount >= 0)
);
```

Payment modes: UPI, Credit Card, Debit Card, Net Banking and Cash on Delivery. Payment statuses: Pending, Success and Failed.

2. Updated ER Diagram (Order–Payment)

Relationship: One order can have zero or one payment record. Payment.Order_ID is a foreign key and is also UNIQUE.

Cardinality: Orders (1) —— Payment (0..1). An order may not yet have a payment row; if a payment row exists, it belongs to exactly one order.

Important: Customer is joined through Orders in the supplied SELECT queries. Its table structure is not included in the uploaded SQL file.

3. Sample Customer, Order & Payment Data

The Payment rows below reproduce the five INSERT records in the SQL script. CURDATE() inserts the date on which the script is executed.

Payment_ID*	Order_ID	Payment Date	Mode	Status	Amount (₹)
1	1	CURDATE()	UPI	Success	1,300.00
2	2	CURDATE()	Credit Card	Pending	2,500.00
3	3	CURDATE()	Debit Card	Failed	800.00
4	4	CURDATE()	Net Banking	Success	1,500.00
5	5	CURDATE()	Cash on Delivery	Pending	600.00

*Payment_ID is generated automatically by AUTO_INCREMENT, assuming a fresh empty table.

Illustrative Order and Customer mapping

The uploaded script does not provide Customer or Orders sample rows. The following mapping is an example only, to demonstrate the joins used in the queries; it is not claimed as data inserted by the uploaded script.

Customer_ID (example)	Order_ID (from Payment)	Payment_ID	Mode	Status	Amount (₹)
101	1	1	UPI	Success	1,300.00
102	2	2	Credit Card	Pending	2,500.00
103	3	3	Debit Card	Failed	800.00
104	4	4	Net Banking	Success	1,500.00
105	5	5	Cash on Delivery	Pending	600.00

The script joins Payment → Orders using Order_ID, then Orders → Customer using Customer_ID.

4. Payment Transaction & Revenue Analysis Reports

The following reports are based on the five inserted Payment rows before the UPDATE statement is run.

Report	Result
Total transactions	5
Successful transactions	2
Pending transactions	2
Failed transactions	1
Successful revenue	■2,800.00
Average transaction amount (all statuses)	■1,340.00
Most used payment mode	Tie: all five modes have 1 transaction each

Revenue by payment mode (successful payments only)

Payment Mode	Successful Revenue (■)
UPI	1,300.00
Net Banking	1,500.00
Credit Card	0.00 (no successful row)
Debit Card	0.00 (failed before UPDATE)
Cash on Delivery	0.00 (pending)

Effect of UPDATE: The script changes Payment_ID = 3 from Failed to Success and sets its Payment_Date to CURDATE(). After that update, successful revenue becomes ■3,600.00 and successful transactions become 3. The average of all transaction amounts remains ■1,340.00.

The 'preferred payment method' query uses ORDER BY Total_Transactions DESC LIMIT 1. Since each mode appears once in this sample, the result can be any one of the tied modes unless a tie-breaker is added.

5. Query Output Screenshots (Workbench-style)

The tables below are clean sample result grids for documentation. They are not actual screenshots captured from MySQL Workbench. Run the SQL in Workbench to capture genuine screenshots from your system.

A. Successful payments — before UPDATE

Payment_ID	Order_ID	Payment_Mode	Payment_Status	Transaction_Amount
1	1	UPI	Success	1300.00
4	4	Net Banking	Success	1500.00

B. Failed and pending payments — before UPDATE

Payment_ID	Order_ID	Payment_Mode	Payment_Status	Transaction_Amount
2	2	Credit Card	Pending	2500.00
3	3	Debit Card	Failed	800.00
5	5	Cash on Delivery	Pending	600.00

C. Total successful revenue — before UPDATE

Total_Revenue
2800.00

D. Payment mode transaction count

Payment_Mode	Total_Transactions
UPI	1
Credit Card	1
Debit Card	1
Net Banking	1
Cash on Delivery	1

5. Query Output Screenshots (continued)

E. Successful revenue by mode — before UPDATE

Payment_Mode	Revenue
UPI	1300.00
Net Banking	1500.00

F. Average transaction amount — all statuses

Average_Amount
1340.00

G. Failed / pending orders

Order_ID	Customer_ID*	Payment_Mode	Payment_Status	Amount
2	102	Credit Card	Pending	2500.00
3	103	Debit Card	Failed	800.00
5	105	Cash on Delivery	Pending	600.00

*Customer_ID values in this joined output are illustrative, not present in the uploaded SQL file.

H. After running the UPDATE query

Metric	Before UPDATE	After UPDATE
Successful transactions	2	3
Failed transactions	1	0
Successful revenue	■2,800.00	■3,600.00

Conclusion

The Payment module records one payment per order, validates payment modes and statuses, and supports transaction tracking and revenue analysis. The SQL script demonstrates status filtering, payment updates, customer payment history, payment-mode counts and successful revenue reports.

Source: user-provided file “dbms task 5(3).sql”.